

Efficient Programming

×

Today's goals



▼

Module 1

Finding Performance Bottlenecks

▼

Module 2

Simple Optimization Techniques

▼

Module 3

Computer Architecture and Vectorization

▼

Module 4

Computer Architecture

^

Memory Architecture and Hierarchy

✔

Memory Architecture

Video • 7 min

✔

Processor Memory Hierarchy

Video • 6 min

✔

Cache and Memory Characteristics of a Compute Node

Video • 6 min

Data Access Scaling - BLAS

✔

Data Access Scaling - BLAS

Video • 6 min

Module Assessments

●

Implementing Tiled Matrix Addition

Programming Assignment • 1h

●

Module Quiz

Graded Assignment • 15 min

Module 5

Parallel and High Throughput Computing

▼

Programming assignment

Implementing Tiled Matrix Addition

Launch lab



What to expect

- Due Sep 18, 11:59 PM IST
- Unlimited attempts

InstructionsMy submissionsDiscussions

In this assignment, you'll implement a tiled version of matrix multiplication to improve cache performance. Tiling breaks large matrices into smaller submatrices (tiles) that fit into the cache, reducing memory access overhead.

How Matrix Multiplication Works

In matrix multiplication, the element at row i and column j of the result matrix C is calculated as the dot product of row i of matrix A and column j of matrix B :

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} \cdot B_{kj}$$

In C++, since matrices are stored as 1D arrays in row-major order, the element at row i , column j is accessed with: `C[i * N + j]`

An non-tiled implementation uses three nested loops:

```
1 for (int i = 0; i < N; ++i)
2   for (int j = 0; j < N; ++j)
3     for (int k = 0; k < N; ++k)
4       C[i*N + j] += A[i*N + k] * B[k*N + j];
5
```

Why Use Tiling?

For large matrices, the non-tiled implementation becomes cache inefficient. As the loops iterate, the CPU keeps fetching data from main memory because the working set doesn't fit in cache.

Tiling (also called blocking) improves cache performance by operating on smaller submatrices (tiles) that are sized to fit into cache. This increases data reuse and reduces cache misses.

Task

Complete the implementation of the following function in `tiled_matrix_mult.cpp`:

```
void tiled_matrix_multiply(double* A, double* B, double* C, int N, int tile_size)
```

Where:

- `A`, `B`, and `C` are pointers to the input and output matrices.
- `N` is the dimensions of the square matrices.
- `tile_size` is the size of the tiles to be used for tiling.

Your function should:

- Divide the matrices into tiles of size `tile_size` x `tile_size`.
- Multiply the tiles and accumulate results into `C`.
- Handle edge cases where `N` is not divisible by `tile_size`.

Hints

- Use **three outer loops** to iterate over tiles.
- Use **three inner loops** to multiply elements within each tile.
- Initialize `C` to zeros before accumulating results.
- Use `min(i + tile_size, N)` to avoid out-of-bounds access for edge tiles.
- Start with the outer loops first, then add inner loops for the tile operations.

Compare Performance

The provided code includes a non-tiled implementation. Once you complete your tiled version:

- Run the program with different `tile_size` values.
- Observe the execution times for each configuration.
- Reflect: Do larger tiles result in faster execution times? Why might very small tiles (e.g., `tile_size=1`) perform worse than the non-tiled implementation?

Submission

Submit your updated `tiled_matrix_mult.cpp` with your completed `tiled_matrix_multiply` function.

Report an issue